

【机试题】2019 大疆嵌入式笔试题 A 卷(附超详细解答)

前不久的大疆嵌入式线上笔试，可能是因为最近只是在做毕设项目，还没有来得及认真系统复习，直接崩了。就凭借着记忆，把一些记得住的笔试题分享一下，作下记录。

整个大疆嵌入式线上笔试，分为选择题（单选题、多选题）、填空题、简答题、编程题。也没有将所有的题目都记得，就分成填空选择题、简答题和编程题三块来介绍吧。

填空选择题

1、ARM 指令和 Thumb 指令。（选择题）

解答：在 ARM 的体系结构中，可以工作在三种不同的状态，一是 ARM 状态，二是 Thumb 状态及 Thumb-2 状态，三是调试状态。而 ARM 状态和 Thumb 状态可以直接通过某些指令直接切换，都是在运行程序，只不过指令长度不一样而已。

ARM 状态：arm 处理器工作于 32 位指令的状态，所有指令均为 32 位；

Thumb 状态：arm 执行 16 位指令的状态，即 16 位状态；

thumb-2 状态：这个状态是 ARM7 版本的 ARM 处理器所具有的新的状态，新的 thumb-2 内核技术兼有 16 位及 32 位指令，实现了更高的性能，更有效的功耗及更少地占用内存。总的来说，感觉这个状态除了兼有 arm 和 thumb 的优点外，还在这两种状态上有所提升，优化；

调试状态：处理器停机时进入调试状态。

也就是说：ARM 状态，此时处理器执行 32 位的字对齐的 ARM 指令；Thumb 状态，此时处理器执行 16 位的，半字对齐的 THUMB 指令。

ARM 状态和 Thumb 状态切换程序：

从 ARM 到 Thumb: LDR R0,=lable+1 BX R0（状态将寄存器的最低位设置为 1，BX 指令、R0 指令将进入 thumb 状态）；

从 ARM 到 Thumb: LDR R0,=lable BX R0（寄存器最低位设置为 0，BX 指令、R0 指令将进入 arm 状态）。

当处理器进行异常处理时，则从异常向量地址开始执行，将自动进入 ARM 状态。

关于这个知识点还有几个注意点：

ARM 处理器复位后开始执行代码时总是只处于 ARM 状态；

Cortex-M3 只有 Thumb-2 状态和调试状态；

由于 Thumb-2 具有 16 位/32 位指令功能，因此有了 thumb-2 就无需 Thumb 了。

另外，具有 Thumb-2 技术的 ARM 处理器也无需再 ARM 状态和 Thumb-2 状态间进行切换了，因为 thumb-2 具有 32 位指令功能。

参考文章：ARM 处理器的工作状态。

2、哪种总线方式是全双工类型、哪种总线方式传输的距离最短？（选择题）

解答：几种总线接口的通信方式的总结如下图所示：

UART、I2C、SPI、USB 异同

总线接口	串/并	同步/异步	速率	工作方式	用线	总线拓扑结构	信距离
UART	串	异步	慢 波特率设置	全双工	2线 Rx、Tx	RS485支持总线式、 星形、树形	远 最远1200m
I2C	串	同步	慢	半双工	2线 SDA、SCL	总线型（特殊的树形）	近
SPI	串	同步	快	全双工	3线或4线 SCLK、SIMO、 SOMI、SS(片选)	环形	远
USB	串	同步	快	半双工	4线 Vbus（5V）、GND、 D+、D-（3.3V）	星形	近

3、TCP 与 UDP 的区别。（选择题）

解答：TCP 和 UDP 的区别总结如下图所示：

TCP 和 UDP 的区别

角度	TCP	UDP
是否连接	面向连接（发送数据前需要建立连接）	无连接（发送数据无需连接）
是否丢包重试	实现了数据传输时各种控制功能，可以进行丢包的重发控制， 还可以对次序乱掉的分包进行顺序控制	不会进行丢包重试，也不会纠正到达的顺序
模式	流模式（面向字节流）	数据报模式（面向报文）
对应关系	一对一	支持一对一，一对多，多对一和多对多的交互通信
头部开销	最小20字节	只有8字节
可靠性	全双工非常可靠、无差错、不丢失、不重复、且按序到达	不保证可靠交付，不保证顺序到达
拥塞控制	有控制 更多详情	有拥塞控制，因此网络出现拥塞不会使源主机的发送速率降低 （对实时应用很有用，如IP电话，实时视频会议等）
资源要求	TCP程序结构较复杂，较多	UDP程序结构简单，少

4、Linux 的用户态与内核态的转换方法。（选择题）

解答：Linux 下内核空间与用户空间进行通信的方式主要有 syscall（system call）、procfs、ioctl 和 netlink 等。

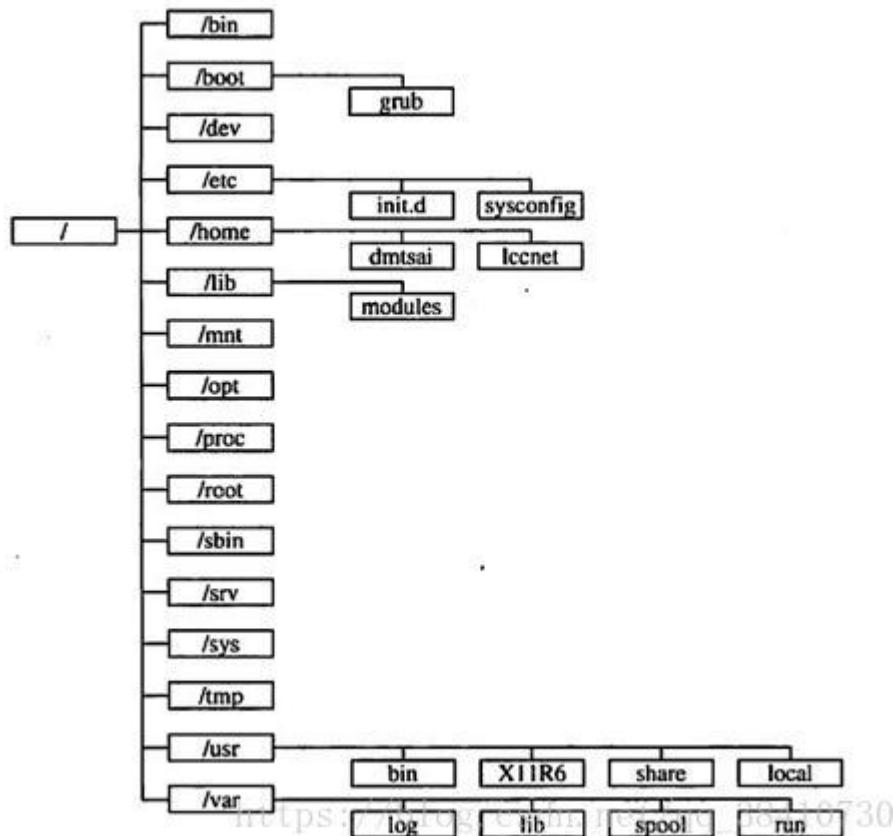
syscall: 一般情况下，用户进程是不能访问内核的。它既不能访问内核所在的内存空间，也不能调用内核中的函数。Linux 内核中设置了一组用于实现各种系统功能的子程序，用户可以通过调用他们访问 linux 内核的数据和函数，这些系统调用接口（SCI）称为系统调用；**procfs:** 是一种特殊的伪文件系统，是 Linux 内核信息的抽象文件接口，大量内核中的信息以及可调参数都被作为常规文件映射到一个目录树中，这样我们就可以简单直接的通过 echo 或 cat 这样的文件操作命令对系统信息进行查取；

netlink: 用户态应用使用标准的 socket API 就可以使用 netlink 提供的强大功能；

ioctl: 函数是文件结构中的一个属性分量，就是说如果你的驱动程序提供了对 ioctl 的支持，用户就可以在用户程序中使用 ioctl 函数控制设备的 I/O 通道。

5、linux 目录结构，选项是 /usr、/tmp、/etc 目录的作用。（选择题）

解答：linux 目录图：



/usr: 不是 user 的缩写，其实 usr 是 Unix Software Resource 的缩写，也就是 Unix 操作系统软件资源所放置的目录，而不是用户的数据啦。这点要注意。FHS 建议所有软件开发者，应该将他们的数据合理的分别放置到这个目录下的次目录，而不要自行建立该软件自己独立的目录；

/tmp: 这是让一般使用者或者是正在执行的程序暂时放置档案的地方。这个目录是任何人都能够存取的，所以你需要定期的清理一下。当然，重要资料不可放置在此目录啊。因为 FHS 甚至建议在开机时，应该要将 /tmp 下的资料都删除；

/etc: 系统主要的设定档几乎都放置在这个目录内，例如人员的帐号密码档、各种服务的启始档等等。一般来说，这个目录下的各档案属性是可以让一般使用者查阅的，但是只有 root 有权力修改。FHS 建议不要放置可执行档(binary)在这个目录中。比较重要的档案有：

/etc/inittab,/etc/init.d/,/etc/modprobe.conf,/etc/X11/,/etc/fstab,/etc/sysconfig/ 等等。

参考文章：Linux 文件目录结构详解。

6、下面这段程序的运行结果？（选择题）

```

1 int main(){
2     const int x=5;
3     const int *ptr;
4     ptr=&x;
5     *ptr=10;
6     printf("%d\n",x);
7
8     return 0;
9 }

```

解答：编译出错。

这道题主要是讲解const与指针的问题：

```

1 const int a;
2 int const a;
3 const int *a;
4 int * const a;
5 const int * const a;
6 int const * const a;

```

- 前两个的作用是一样，a是一个常整型数；
- 第三个意味着a是一个指向常整型数的指针（也就是，整型数是不可修改的，但指针可以）；
- 第四个意思a是一个指向整型 数的常指针（也就是说，指针指向的整型数是可以修改的，但指针是不可修改的）；
- 最后两个意味着a是一个指向常整型数的常指针（也就是说，指针指向的整型数 是不可修改的，同时指针也是不可修改的）。

也就是说：本题x是一个常量，不能改变；ptr是一个指向常整型数的指针。而当*ptr=10;的时候，直接违反了这一点。同时记得一点，const是通过编译器在编译的时候执行检查来确保实现的。

7、在32位系统中，有如下结构体，那么sizeof（fun）的数值是（ ）

```

1 #pragma pack(1)
2
3 struct fun{
4     int i;
5     double d;
6     char c;
7 };

```

解答：13。

可能是一般的内存对齐做习惯了，如果本题采用内存对齐的话，结果就是 24（int4double char 7）。但是#pragma pack(1)让编译器将结构体数据强制按 1 来对齐。

每个特定平台上的编译器都有自己的默认“对齐系数”（32 位机一般为 4，64 位机一般为 8）。我们可以通过预编译命令#pragma pack(k)，k=1,2,4,8,16 来改变这个系数，其中 k 就是需要指定的“对齐系数”。

只需牢记：

第一个数据成员放在 **offset** 为 0 的地方，对齐按照对齐系数和自身占用字节数中，二者比较小的那个进行对齐；

在数据成员完成各自对齐以后，**struct** 或者 **union** 本身也要进行对齐，对齐将按照对齐系数和 **struct** 或者 **union** 中最大数据成员长度中比较小的那个进行；

参考文章：[#pragma pack\(\)的解读](#)。

8、Linux 中的文件/目录权限设置命令是什么？（选择题）

解答：chmod

9、下面四个选项是四个整数在内存中的存储情况，请选择其中最大的一个。（选择题）

四个整数在内存的存储情况			
A	B	C	D
Big-endian 低地址 高地址 12 34 56 78	Big-endian 低地址 高地址 56 78 12 34	Little-endian 低地址 高地址 34 56 78 12	Little-endian 低地址 高地址 78 12 34 56

解答：大端小端问题：

- 所谓的大端模式（BE big-endian），是指数据的低位保存在内存的高地址中，而数据的高位，保存在内存的低地址中（低对高，高对高）；
- 所谓的小端模式（LE little-endian），是指数据的低位保存在内存的低地址中，而数据的高位保存在内存的高地址中（低对低，高对高）。

那么A：12345678、B：56781234、C：12785634、D：56341278。

10、C语言的各种变量的存取区域，给你一段小程序，让你分析各个变量的存储区域（填空题）

解答：具体的题目内容忘了，但是大体上给出各个变量可能的存储区域：

- 堆：堆允许程序在运行时动态地申请某个大小的内存。一般由程序员分配释放；
- 栈：由编译器自动分配释放，存放函数的参数值，局部变量等值；
- 静态存储区：一定会存在且不会消失，这样的数据包括常量、常量（const 变量）、静态变量、全局变量等；
- 常量存储区：常量占用内存，只读状态，决不可修改，常量字符串就是放在这里的。

11、下面这段程序的运行结果？（填空题）

```
1 int main() {  
2     int a[10] = { 0,1,2,3,4,5,6,7,8,9 };  
3     memcpy(a + 3, a, 5);  
4     for (int i = 0; i<10; i++){  
5         printf("%d ", a[i]);  
6     }  
7     return 0;  
8 }
```

登录后复制

解答：0 1 2 0 1 5 6 7 8 9